

Question Bank — Week 6

Stacks (Lecture 06) — Answers

DSA27 · Data Structures and Algorithms

Fall 2026

Questions: [week06-questions.md](#). Commit to your own answer before reading one here.

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	a	M07	b	M13	a	M19	c
M02	c	M08	d	M14	c	M20	d
M03	b	M09	a	M15	b	M21	a
M04	d	M10	c	M16	d	M22	b
M05	a	M11	b	M17	a		
M06	c	M12	d	M18	b		

W6-M01 — a. First in, first out (b) is a queue — Week 7.

W6-M02 — c. The stack goes $[1, 2, 3] \rightarrow \text{pop } 3 \rightarrow [1, 2] \rightarrow \text{push } 4 \rightarrow [1, 2, 4] \rightarrow \text{pop } 4 \rightarrow [1, 2]$; the next pop returns 2.

W6-M03 — b. append and pop() at the end move nothing. Index 0 (a) shifts everything on every operation.

W6-M04 — d. push_front and pop_front are $O(1)$. At the last node (a), popping would need the node before it — an $O(n)$ walk.

W6-M05 — a. The contract says IndexError, as list.pop() does. Returning None (b) lets an error travel silently.

W6-M06 — c. Inserting at index 0 of an array shifts every element — the “dishonest” design that still passes every test.

W6-M07 — b. The partner check fails. The count is even (a), and the stack is not empty at that point (d).

W6-M08 — d. The closer finds nothing to match. The partner check (a) cannot even run.

W6-M09 — a. * binds tighter, so $4 \cdot 2$ * is computed first, then added to 3.

W6-M10 — c. The parentheses group $3 \cdot 4 +$ first; the result is then multiplied by 2.

W6-M11 — b. The first value popped (3) is the **right** operand, the second (8) the **left**: $8 - 3$.

W6-M12 — d. $3 \cdot 4$ is 12, then $2 \cdot 12$ is 24.

W6-M13 — a. Higher **or equal**. Strictly higher (d) is the rule for right-associative operators such as $**$.

W6-M14 — c. Popping the earlier - before pushing the later one computes the left subtraction first.

W6-M15 — b. Left associative: $(8 - 3) - 2$. $8 - 3 - 2$ - (a) is $8 - (3 - 2)$, which is 7, not 3.

W6-M16 — d. A text of n openers, $(((((...$, pushes all n .

W6-M17 — a. The call stack does the bookkeeping for a recursion; without it, you keep that stack yourself.

W6-M18 — b. Redo only makes sense for actions just undone; a new action starts a new history.

W6-M19 — c. Each push is an append to the `DynamicArray`: amortised $O(1)$.

W6-M20 — d. First come, first served is FIFO — a queue.

W6-M21 — a. Four openers in a row, then four closers.

W6-M22 — b. And, like pop, it raises `IndexError` on an empty stack — so (d) is wrong.

Part B — Short answer and essay

W6-E1 (4)

- **ADT:** `push(x)`, `pop()`, `peek()`, `is_empty()`, `len()` — LIFO, all $O(1)$; pop and peek raise `IndexError` when empty.
- **On a dynamic array**, top at the end: push is amortised $O(1)$ (occasional resize), pop and peek $O(1)$; compact memory, contiguous.
- **On a linked list**, top at the head: all worst-case $O(1)$; a node per item (about six times the memory of an array slot).
- **Dishonest:** top at index 0 of an array — every push and pop shifts all the elements, $O(n)$ each, $O(n^2)$ for n operations — yet the stack behaves correctly and passes every test.

W6-E2 (4)

- Scan the text. **Opener:** push. **Closer:** the stack must be non-empty and the popped opener must be its partner. **End:** the stack must be empty. Other characters are ignored.
- **Wrong partner:** `([)]` — `)` pops `[`.
- **Closer with nothing open:** `)` or `)` — the stack is empty when a closer arrives.
- **Opener never closed:** `(` or `((` — the stack is not empty at the end.
- $O(n)$ time, $O(n)$ space in the worst case.

W6-E3 (3)

- Scan the tokens: push numbers; for an operator, pop two values, apply it, push the result; at the end exactly one value must remain (otherwise the input was malformed).
- The **first** value popped is the **right** operand and the **second** the **left**, because the left operand was pushed earlier.
- It matters for the non-commutative operators, - and /: $8 \ 3 \ -$ must be $8 - 3 = 5$, not $3 - 8 = -5$.

W6-E4 (4)

- Output list and operator stack. Numbers go to the output; (is pushed;) pops to the output down to the matching (; an operator pops every stacked operator of higher **or equal** precedence, then is pushed; at the end the stack is emptied to the output.
- **Trace** of $2 * (3 + 4) - 5$:

Token	Output	Stack
2	2	
*	2	*
(	2	* (
3	2 3	* (
+	2 3	* (+
4	2 3 4	* (+
)	2 3 4 +	*
-	2 3 4 + *	-
5	2 3 4 + * 5	-
end	2 3 4 + * 5 -	

- **Equal precedence pops** so that left-associative operators apply left to right: in $8 - 3 - 2$, the first - must be applied before the second.

W6-E5 (3)

- Every call pushes a frame (its local variables and where to return) on the call stack, and every return pops one — so a recursion is a computation driven by a stack.
- **Rewrite**: push the initial problem on an explicit Stack; loop while it is not empty: pop a problem, solve it directly if it is a base case, otherwise push its sub-problems.
- **Worth it in Python** when the recursion could go deeper than about 1,000 frames (RecursionError), since there is no tail-call optimisation — e.g. a depth-first search on a large graph (Week 14).

Part C — Trace the code and the algorithms

W6-T1

[2, 4, 4, 3, 1]

After pop → 2 the stack is [1]; push 3, 4 → [1, 3, 4]; peek → 4 (not removed); pop → 4; pop → 3; one item, 1, is left.

W6-T2

Token	Stack (top on the right)
5	5
1	5 1
2	5 1 2
+	5 3
4	5 3 4
*	5 12
+	17
3	17 3
-	14

The answer is **14** — the expression $5 + (1 + 2) * 4 - 3$.

W6-T3. The table in W6-E4: the result is **2 3 4 + * 5 -**.

W6-T4.

Character	Stack after it
{	{
[	{ [
(	{ [(
)	{ [
(	{ [(
)	{ [
]	{
}	(empty)

Balanced — the stack is empty at the end. Maximum size **3**.

W6-T5

4 3 2 1

The inner items of a list are pushed left to right, so the **last** one is popped first: 4, then the list [2, 3], whose items are pushed 2, 3 and popped 3, 2; then 1. A stack reverses the order things were pushed in. To visit left to right, push the items in reverse — exactly what `dfs_iterative` does in Week 14.

Part D — Find and fix the bug

W6-B1. An empty pop returns `None` instead of raising `IndexError`, breaking the contract (and `test_stack_empty_behaviour`). Worse, `None` can then be pushed, compared or printed far from the real mistake. **Fix:** `raise IndexError("pop from empty stack")`.

W6-B2. It never checks that the stack is **empty at the end**, so unclosed openers pass: `is_balanced("(")` returns `True`. **Fix:** `return s.is_empty()`.

W6-B3. The first value popped is the **right** operand, but it is passed as the left: `8 3 -` gives `3 - 8 = -5`, and `8 2 /` gives `0.25`. Addition and multiplication hide the bug. **Fix:** `right = stack.pop(); left = stack.pop(); stack.push(apply(token, left, right))`.

W6-B4. Popping only on **strictly** higher precedence makes `-` and `/` right associative: `8 - 3 - 2` becomes `8 3 2 - -`, which evaluates to `7` instead of `3`. **Fix:** `>=`.

Part E — Write the code

W6-C1

```
def reverse_words(sentence):
    words = Stack(sentence.split())
    out = []
    while not words.is_empty():
        out.append(words.pop())
    return " ".join(out)
```

Pushing and popping reverses the order — the stack's defining property.

W6-C2

```
class MinStack:
    def __init__(self):
        self._values = Stack()
        self._mins = Stack()           # _mins.peek() is the minimum so far

    def push(self, value):
        self._values.push(value)
        if self._mins.is_empty() or value < self._mins.peek():
            self._mins.push(value)
        else:
            self._mins.push(self._mins.peek())

    def pop(self):
        self._mins.pop()               # IndexError when empty, as required
        return self._values.pop()

    def peek(self):
        return self._values.peek()

    def get_min(self):
        return self._mins.peek()
```

```
def __len__(self):
    return len(self._values)
```

Why one variable is not enough: when the minimum is popped, you need the minimum of what is left — which you could only find again by searching, $O(n)$. The second stack remembers the minimum at every level, so after any pop the new minimum is simply on top.

W6-C3

```
def next_greater(values):
    result = [-1] * len(values)
    waiting = Stack() # indices still waiting for an answer
    for i, value in enumerate(values):
        while not waiting.is_empty() and values[waiting.peek()] < value:
            result[waiting.pop()] = value
        waiting.push(i)
    return result
```

Why linear: the inner while does not run up to n times per iteration every time. Each index is pushed exactly once and popped at most once, so across the whole run the inner loop body executes at most n times in total: $O(n)$. This is **aggregate** counting — the same argument as amortised analysis in Lecture 04. (The stack always holds values in decreasing order — a *monotonic stack*.)

W6-C4

```
def tags_balanced(html):
    open_tags = Stack()
    i = 0
    while True:
        start = html.find("<", i)
        if start == -1:
            break
        end = html.find(">", start)
        if end == -1:
            return False
        tag = html[start + 1:end]
        if tag.startswith("/"):
            if open_tags.is_empty() or open_tags.pop() != tag[1:]:
                return False
        else:
            open_tags.push(tag)
        i = end + 1
    return open_tags.is_empty()
```

The same three checks as brackets, with tag names instead of bracket characters.

W6-C5

```
def decode(encoded):
    counts, texts = Stack(), Stack()
    current, number = "", 0
    for ch in encoded:
        if ch.isdigit():
            number = number * 10 + int(ch)
        elif ch == "[":
            counts.push(number)
            texts.push(current)          # save what came before this group
            current, number = "", 0
        elif ch == "]":
            current = texts.pop() + current * counts.pop()
        else:
            current += ch
    return current
```

On [, the text so far and the repeat count are saved and a fresh group starts; on], the group is repeated and appended to what was saved. Nesting works because the innermost group is always on top — exactly as with brackets.